

ENTREGA DEL RETO

Storefront Change Guard

Documento de decisiones: alcance, diseño, criterios y evolución del prototipo

Documento de decisión técnica

Junio de 2026

Resumen ejecutivo. Diseñé Storefront Change Guard para asistir la revisión de cambios, validar una corrección propuesta y determinar el estado de readiness con evidencia verificable. Mi objetivo no fue automatizar todo el SDLC, sino demostrar un flujo local-first, acotado y auditable de punta a punta, donde la IA aporta análisis y el sistema conserva la autoridad operativa y decisoria.

1. Alcance que decidí abordar

Decidí concentrarme en una etapa donde los equipos de e-commerce suelen perder más tiempo y confianza: revisar cambios de código, verificar que una corrección realmente resuelva el problema y determinar si el cambio cuenta con evidencia suficiente para avanzar. Elegí este alcance porque combina valor técnico concreto con límites claros y auditables.

En lugar de intentar automatizar todo el ciclo de desarrollo, preferí demostrar un flujo completo y controlado. Las capacidades que incorporé son:

1. Revisar cambios de código y devolver hallazgos accionables.
2. Detectar un problema y validar una corrección suministrada.
3. Responder preguntas acotadas sobre código o documentación.
4. Determinar readiness mediante criterios fijos y evidencia previa.

El reto exige al menos dos capacidades. Incluí cuatro, pero las mantuve deliberadamente limitadas por reglas de seguridad: el modelo no ejecuta comandos, no modifica el repositorio y no decide readiness.

2. Diseño del sistema y separación de autoridad

Diseñé el sistema como un flujo local-first con responsabilidades separadas. Cada etapa recibe una entrada concreta, produce artefactos verificables y deja evidencia reutilizable para la siguiente.

01 · Intake	02 · Evidencia	03 · Análisis	04 · Contrato	05 · Worktree	06 · Gates	07 · Política
Entrada determinística y alcance definido.	Git y archivos solicitados, sin acceso implícito adicional.	Salida local y estructurada del modelo.	Validación determinística del JSON recibido.	Parche suministrado, aislado del checkout principal.	Comandos limitados por allowlist.	READY, NOT_READY o INSUFFICIENT_EVIDENCE.

La IA analiza; el sistema valida, restringe y decide.

El modelo local solo produce análisis estructurado. El resto del sistema valida, rechaza o toma decisiones mediante código determinístico. Esta separación es esencial: una revisión asistida por IA puede ser útil, pero no debe tener permisos para ejecutar shell, aplicar parches, hacer merge ni aprobar producción.

Usé un storefront pequeño y controlado porque el valor del reto está en demostrar el uso de evidencia real, no en el tamaño del repositorio base. El escenario de shipping y montos en centavos ofrece una regla de negocio concreta, pruebas reproducibles y errores que una audiencia no técnica también puede comprender.

3. Decisiones tomadas y alternativas descartadas

Confiabilidad del contrato antes que velocidad

Priorizé la confiabilidad del contrato por encima del rendimiento bruto. El Test 1 mostró que el modelo 4B era más rápido, pero en el flujo real produjo JSON incompleto bajo el contrato estricto. Por eso cambié a **Qwen3.5-9B-UD-IQ3_XXS.gguf**, que superó los gates A-D con la misma evidencia y el mismo runner.

El Test 1 sigue siendo evidencia válida

No considero que el cambio de modelo invalide el Test 1. Al contrario: documenta que primero medí rendimiento y luego ajusté la elección cuando apareció un problema real de operabilidad. Esa trazabilidad refleja una decisión basada en evidencia, no una optimización retrospectiva.

Inferencia local, no cloud-only

Descarté una arquitectura cloud-only. Habría simplificado la instalación, pero el código de una empresa puede ser sensible y quise demostrar privacidad local y control sobre el entorno de ejecución.

Menos complejidad operativa

Descarté el enrutamiento entre múltiples modelos y los re-intentos automáticos porque añadían complejidad y costo sin aportar valor proporcional al prototipo. También descarté que el modelo genere comandos de shell y descarté el auto-merge: validar una corrección no equivale a autorizar una integración.

4. Tensiones del reto y balance adoptado

Costo y privacidad favorecen inferencia local. **Tiempo de respuesta** favorece cloud o modelos más pequeños. **Operabilidad** exige una instalación clara y artefactos reproducibles. Mi balance fue usar un único modelo local, un alcance explícito, caché segura, evidencia acotada y fases determinísticas en los puntos críticos. Así reduzco llamadas al modelo y mantengo la decisión final fuera de la IA.

Límite de seguridad principal.

Phase 04 aplica un parche suministrado únicamente en un worktree separado; el checkout principal no se modifica. Phase 05 no ejecuta pruebas nuevas ni llama al modelo: solo lee artefactos JSON previos y aplica la política de readiness.

5. Qué haría distinto con más tiempo

Extendería el prototipo sin alterar sus límites de autoridad. Las mejoras prioritarias serían:

- Una interfaz visual mínima para cargar una PR, inspeccionar evidencia, revisar hallazgos y aprobar manualmente una acción.
- Soporte para más tipos de proyectos e-commerce y una suite mayor de fixtures con errores intencionales.
- Comparación automática de calidad entre modelos y exportación de reportes HTML.
- Para un contexto productivo: autenticación, permisos por rol, firma de artefactos, trazabilidad por PR y cola de ejecuciones.

6. Uso de IA y autoría propia

Usé IA como apoyo para redacción, revisión, organización de etapas, prompts de trabajo, explicaciones y chequeos de consistencia. Sin embargo, las decisiones de alcance, el foco en review/validación/readiness, la separación de autoridad, la trazabilidad por fases, el cambio de modelo frente a fallas reales del contrato y el cierre de la entrega fueron decisiones mías.

La IA fue una herramienta de apoyo. No delegué la comprensión del sistema ni la responsabilidad sobre el entregable.